FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 8

Efficiency: quantization, distillation, low-rank adaptation, MoE.

Zdeněk Kasner

31 Mar 2026

Why efficiency matters

The cost of running LLMs

Modern LLMs are expensive to **train**, **store**, and **run**:

- **Training**: GPT-4 cost an estimated $100M+ in compute.
- **Inference**: serving billions of queries per day requires massive GPU clusters.
- **Storage**: a 70B model in float32 takes $\approx$ 280 GB of disk space.
- **Memory**: loading even a 7B model requires $\approx$ 14 GB of GPU memory.

Question

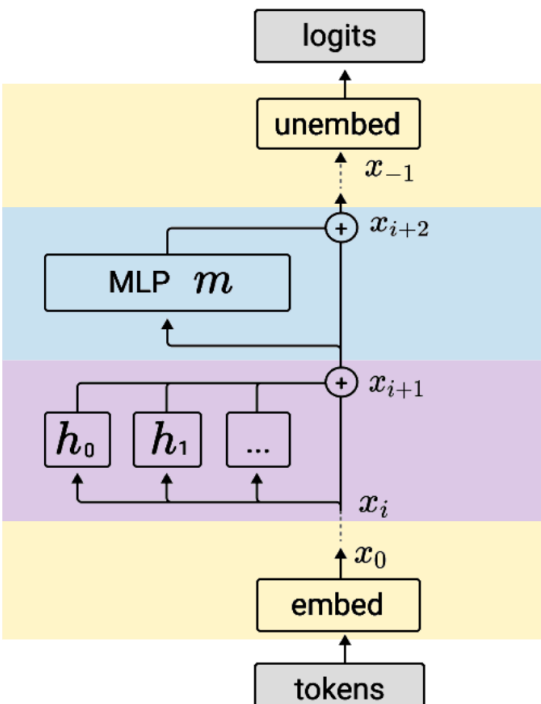
Can you think of ways how to make LLMs smaller, faster, and cheaper (without sacrificing too much performance?)

Transformer: algorithmic complexity

Variables: hidden state dimension (D), sequence length (N), vocabulary size (V).

	Time	Space
final projection (unembedding)	$N \cdot V \cdot D$ each state projected to logits over the vocabulary	$V \cdot D + V \cdot N$ (un)embedding matrix 📦 output logits 🧮
MLP	$N \cdot D^2$ projection matrix 📦	$D^2 + N \cdot D$ activations 🧮
multi-head attention	$N \cdot D^2 + N^2 \cdot D$ queries, keys, values attention matrix 📦	$N^2 + N \cdot D + D^2$ projected values 🧮 Q, K, V matrices 📦
embedding	$N \cdot D$ each token is embedded	$V \cdot D + N \cdot D$ embedding matrix 📦 embeddings 🧮

📦 = model parameters, 🧮 = computed values



source: Transformer circuits

LLM bottlenecks: computational throughput

Time complexity

$$\underbrace{N \cdot D^2}_{\text{MLP \& projections}} + \underbrace{\boldsymbol{N^2}}_{\text{attention matrix}} \cdot \boldsymbol{D} + \underbrace{N \cdot V \cdot D}_{\text{unembedding}}$$

long context → $N^2 \gg D^2$

happens only once

Computing **multi-head attention** becomes the main bottleneck when $N^2 \gg D^2$ (long context).

Solution: efficient attention, MoE, distillation.

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

LLM bottlenecks: disk space

Space complexity: model parameters 📦

embedding matrix *MLP & projections*

$$V \cdot D + D^2$$

stored only once

MLPs typically take up the majority of model parameters – they are repeated in each layer and the multiplicative factor is larger than for attention.

Solution: parameter quantization, (Q)LoRA, distillation.

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

LLM bottlenecks: memory

Space complexity: activations 🧮

$$\underbrace{\boldsymbol{N^2}}_{\text{attention matrix}} + \underbrace{N \cdot D}_{\text{hidden states}} + \underbrace{N \cdot V}_{\text{logits}}$$

gets more important
with growing input context size

Activations take extra space in GPU memory (alongside the model parameters) when the model is performing a computation.

Solution: parameter quantization, (Q)LoRA, distillation

Typical hyperparameter values:

Param	Min	Max
N	512	128,000+
D	768	4,096+
V	30,000	100,000+
blocks	8	128+

Building an intuition about sizes

Question

How much memory does it approximately take to store a **7B model** (=7 billion parameters) in **fp16 precision**, meaning each parameter is stored as a 16-bit floating point number?

Each parameter takes 2 bytes (16 bits), so:

7 billion parameters · 2 bytes per parameter = 14B bytes = **14 GB**.

LLM bottlenecks: training vs. inference

In terms of resource requirements: **pre-training** $\gg$ **finetuning** $\gg$ **inference**.

However, we typically want to run inference on consumer hardware → it also matters.

Dataset size

- Pre-training: 10T tokens $\approx$ 50 TB.
- Finetuning: up to 100s GB.

Memory requirements

- Inference: 2 bytes per parameter → 14 GB for a 7B model.
- Training / finetuning: 2 bytes per parameter + 2 bytes per gradient + 12 bytes per optimizer state (Adam) → **112 GB** for a 7B model.

Efficient algorithms: overview

We will go over the following techniques:

Algorithm	What it targets	Train. memory	Train. speed	Inf. memory	Inf. speed
Quantization	Reducing param. size	–	–	✓✓	✓
Distillation	Reducing # of params	–	–	✓✓	✓✓
MoE	Reducing active params	–	✓✓	–	✓
Linear attention	Faster attention using math tricks	✓	✓	✓	✓
FlashAttention	Faster attention using HW optimizations	✓✓	✓	✓	✓✓
(Q)LoRA	Efficient finetuning	✓✓	✓	–	–

Quantization

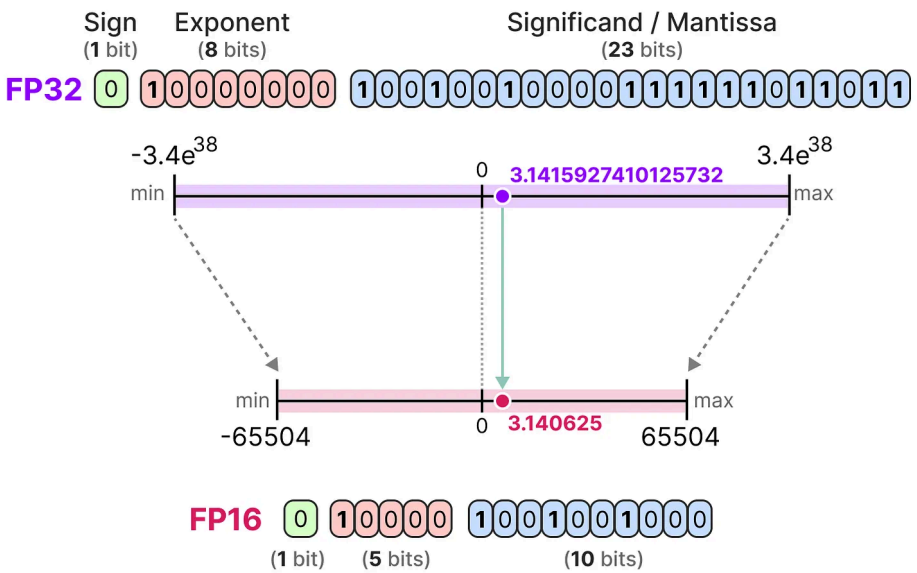
Bits per parameter

source: Grootendorst (2024)

Model parameters are floating point numbers. How do we store them? And can we store them more efficiently?

Floating point numbers

- `float64` – native Python (→64 bits).
- `float32` – baseline precision (→32 bits).
- `float16` – half precision (→16 bits).



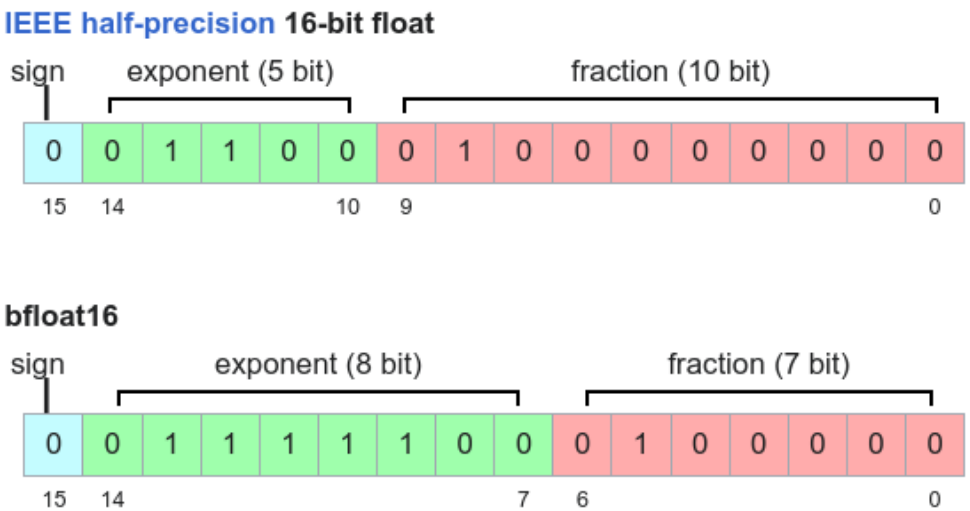
- Inference: **little performance degradation** with `float16` → used with LLMs.
- Training: `float16` can lead to vanishing gradients → **mixed-precision training**.

bfloat16

source: Wikipedia

`bfloat16`: Developed by Google Brain (→ “brain float”).

- A 16-bit float **optimized for ML models**.
- Greater **dynamic range** than float16 (supports outlier weights) at the cost of lower precision.
- Now the **standard format** for training and inference in most LLM frameworks.



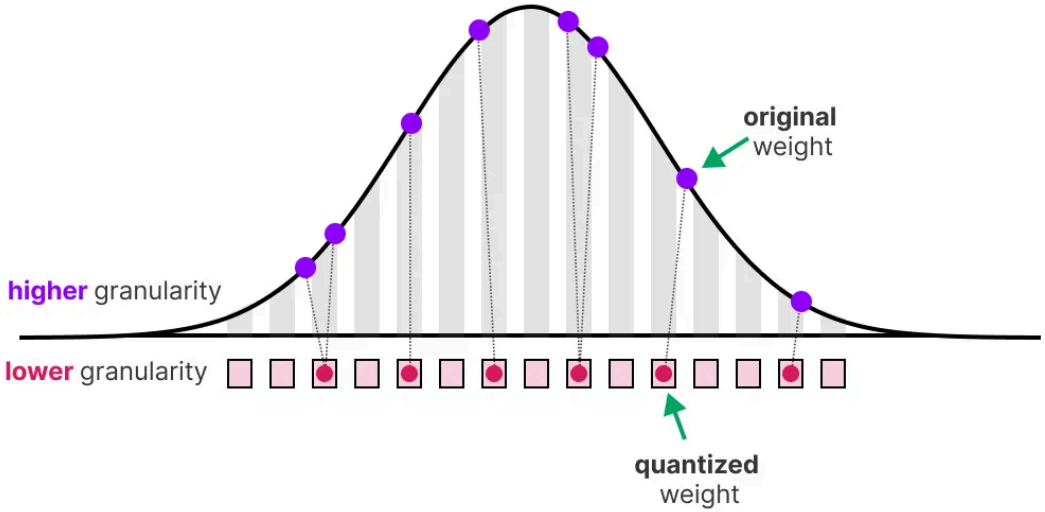
Parameter quantization

source: Grootendorst (2024)

Can we go beyond a 16-bit float?

Idea

Let's use `int8` having **8-bit per parameter** → we'll save twice as much space.



Parameter quantization

source: Grootendorst (2024)

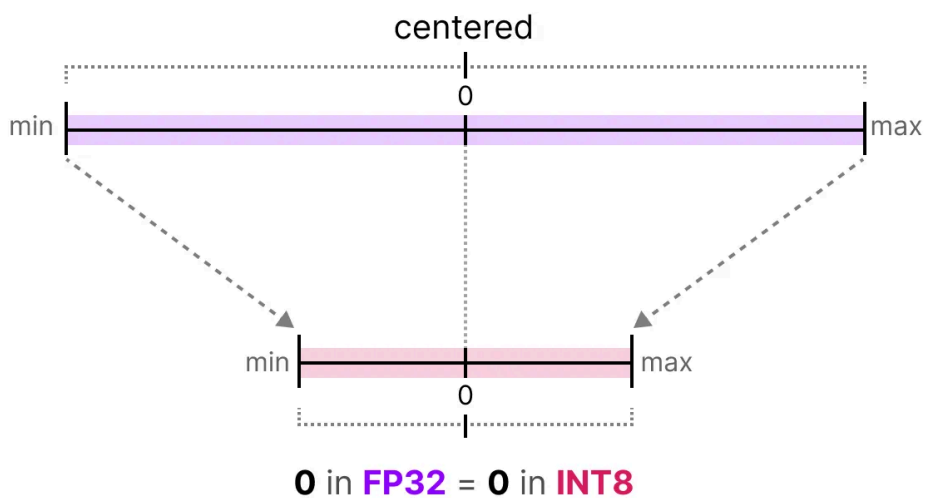
Question

The range of `int8` is $(-127, 127)$, while floats have a huge dynamic range ($\approx 10^{-38}$ to 10^{38}). How do we squeeze floating-point weights into the int range?

We can find the **maximum absolute value** and scale all weights proportionally:

$$x_{\text{quant}} = \text{round}\left(\frac{127}{\max(|x|)} \cdot x\right)$$

We only use the range of our actual model weights → the range is not that wide.

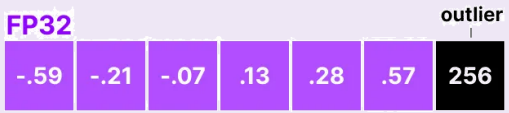


Parameter quantization

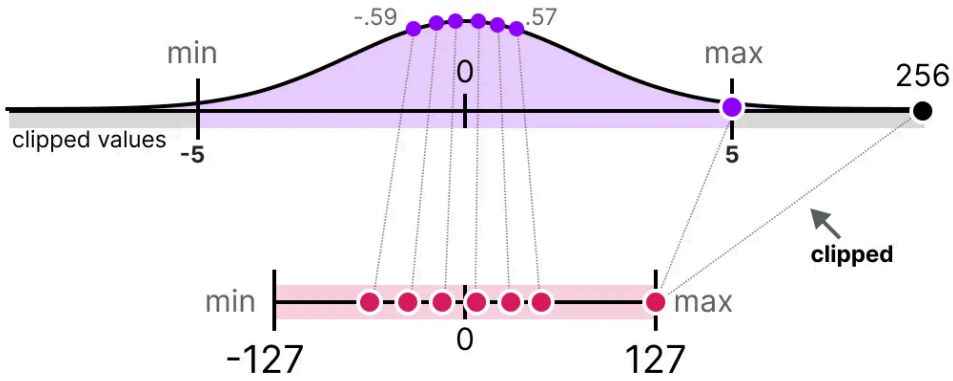
source: Grootendorst (2024)

Question

What if some of the weights are **outliers**? This would map most of the weights to zero (or near zero):



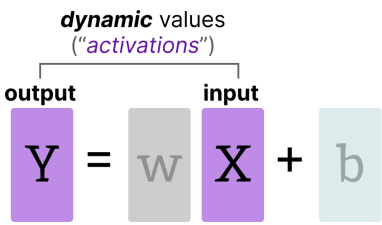
Simple solution: **clip the outliers**:



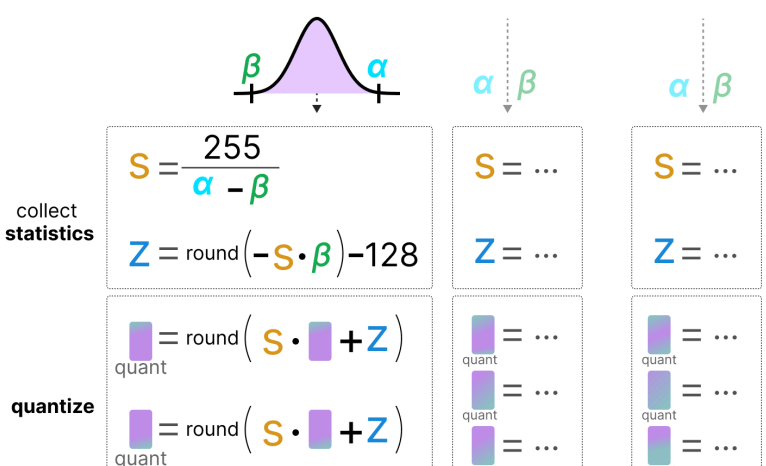
Parameter quantization

source: Grootendorst (2024)

Model parameters can be quantized offline, but activations change with every input.



Solution: compute the quantization range **dynamically** at inference time:



Extreme quantization: 1-bit models

source: Ma et al. (2024)

Can we go all the way down to **1-bit**?

BitNet: Scaling 1-bit Transformers for Large Language Models

Hongyu Wang*†‡ Shuming Ma*† Li Dong† Shaohan Huang†
Huaijie Wang§ Lingxiao Ma† Fan Yang† Ruiping Wang‡ Yi Wu§ Furu Wei†◇
† Microsoft Research ‡ University of Chinese Academy of Sciences § Tsinghua University
https://aka.ms/GeneralAI

BitNet (Ma et al., 2024): Each weight is either −1 or +1.

Dot product reduced to addition → massive speedup on specialized hardware.

Post-training quantization in practice

GPTQ (Frantar et al., 2022)

- Algorithm for layer-wise quantization.
- Optimized for GPU inference.
- More robust for extreme quantization (→ down to 2-bit).
- Used with HF transformers.

GGUF

- File format based on block-wise quantization.
- Optimized for CPU inference.
- Originally from llama.cpp, used by Ollama.

Note

Models in Ollama are quantized by default to 4-bit.

Quantization: summary

Format	Bits	7B model size	Notes
`float32`	32	$\approx$ 28 GB	Rarely used with LLMs
`float16`	16	$\approx$ 14 GB	Inference with older GPUs (before $\approx$2020, like V100s)
`bfloat16`	16	$\approx$ 14 GB	**Standard for training & inference**
`int8`	8	$\approx$ 7 GB	Good accuracy, 2× savings
`int4`	4	$\approx$ 3.5 GB	Popular for local deployment
`1-bit`	1	$\approx$ 0.9 GB	Experimental, requires special training

Rule of thumb

Going from `float16` to `int4` reduces model size by $\approx$ 4× with often less than 5% performance drop on standard benchmarks.

Knowledge distillation

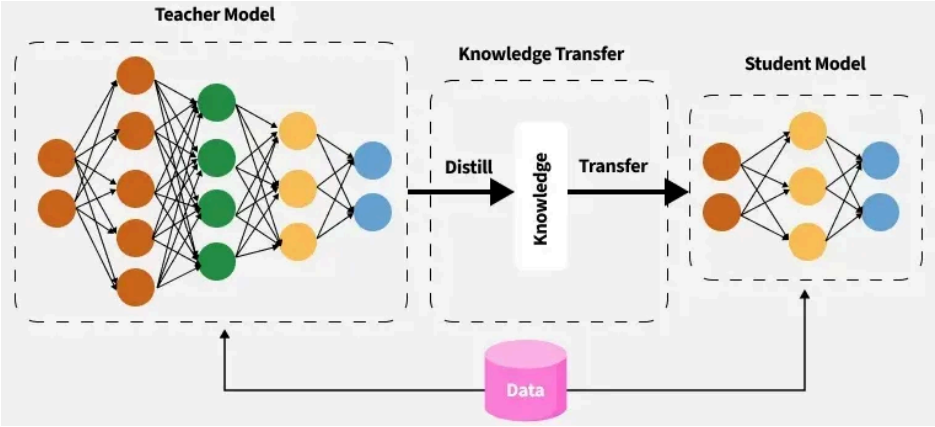
What is knowledge distillation?

source: https://www.geeksforgeeks.org/nlp/what-is-llm-distillation/

We have a **large model** and we want to make it more **efficient**.

Idea: knowledge distillation

We can train a smaller model (**"student"**) on the knowledge of the large model (**"teacher"**).



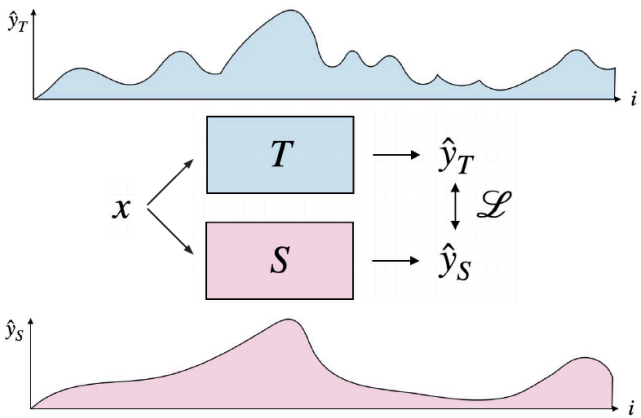
What is knowledge distillation?

source: CME295

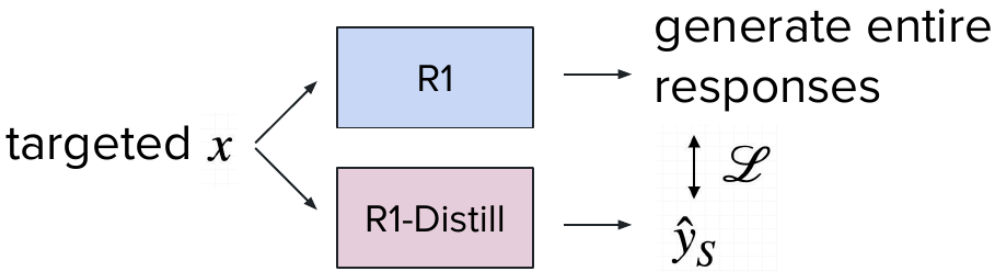
⚠️ Warning

"Knowledge distillation" has two distinct meanings in the context of LLMs.

1. Learning from teacher's distribution:



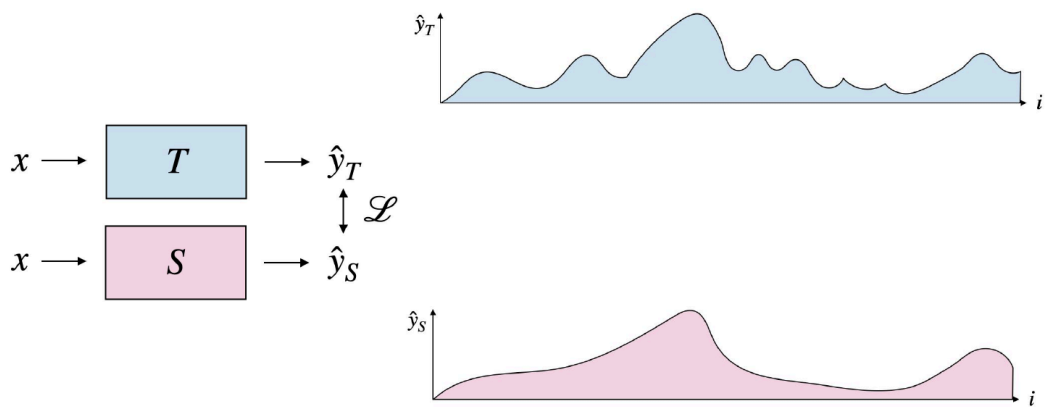
2. Learning from teacher's outputs:



Learning from teacher's distribution

source: CME295

Learning from teacher's distribution: The "original" knowledge distillation introduced by Hinton et al. (2015) for classification models.



The student learns from the teacher's output probability distribution (**soft outputs**) that carry **richer information than hard labels**.

Learning from teacher's distribution

The student is trained using a **KL-divergence loss** between their distribution $\hat{y}_S$ and the teacher's distribution $\hat{y}_T$:

$$\mathrm{KL}(\hat{y}_T \parallel \hat{y}_S) = \sum_i \hat{y}_T^{(i)} \log \left(\frac{\hat{y}_T^{(i)}}{\hat{y}_S^{(i)}} \right)$$

Example

- **Hard label** (ground truth): [cat: 1.0, dog: 0.0, car: 0.0]
- **Soft label** (teacher distribution): [cat: 0.7, dog: 0.2, car: 0.1]

The soft label carries more signal. It tells the student that "a cat is somewhat similar to a dog, but not to a car".

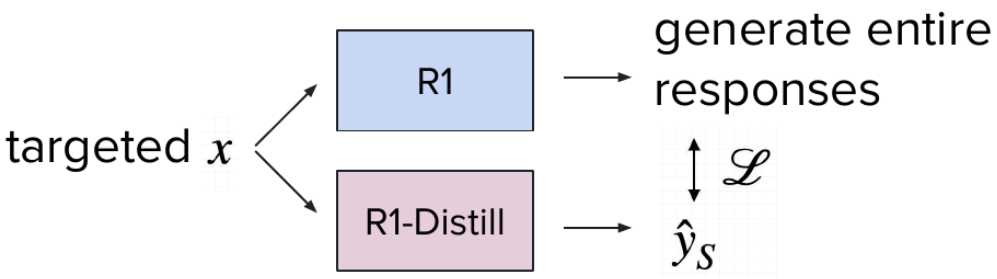
Learning from teacher's outputs

source: CME295

Sometimes we only have access to the teacher's generated text.

In that case, we still can:

- **Generate a large dataset** using the teacher model.
- **Finetune the student** on this synthetic dataset.



Use in practice

This technique was used e.g. to make smaller DeepSeek-R1 models. Also used for training smaller open models on the outputs from larger commercial models.

Finetuning & low-rank adaptation

Finetuning and when it helps

Question

In which cases we may want to finetune a model on our own?

→ To get outputs in a **consistent format** without extensive prompting.

→ To make the model learn our own **domain or a private knowledge base**.

→ To get a **smaller, more efficient** model (→ knowledge distillation).

⚠️ **But finetuning is computationally expensive**

We saw that finetuning a 7B model requires ≈ **112 GB** of GPU memory. For comparison, a single H100 (96GB GPU) costs around $25,000.

Low-rank adaptation (LoRA)

source: Hu et al. (2021)

The issue is alleviated by **LoRA** (Hu et al., 2021) : low-rank adaptation method.

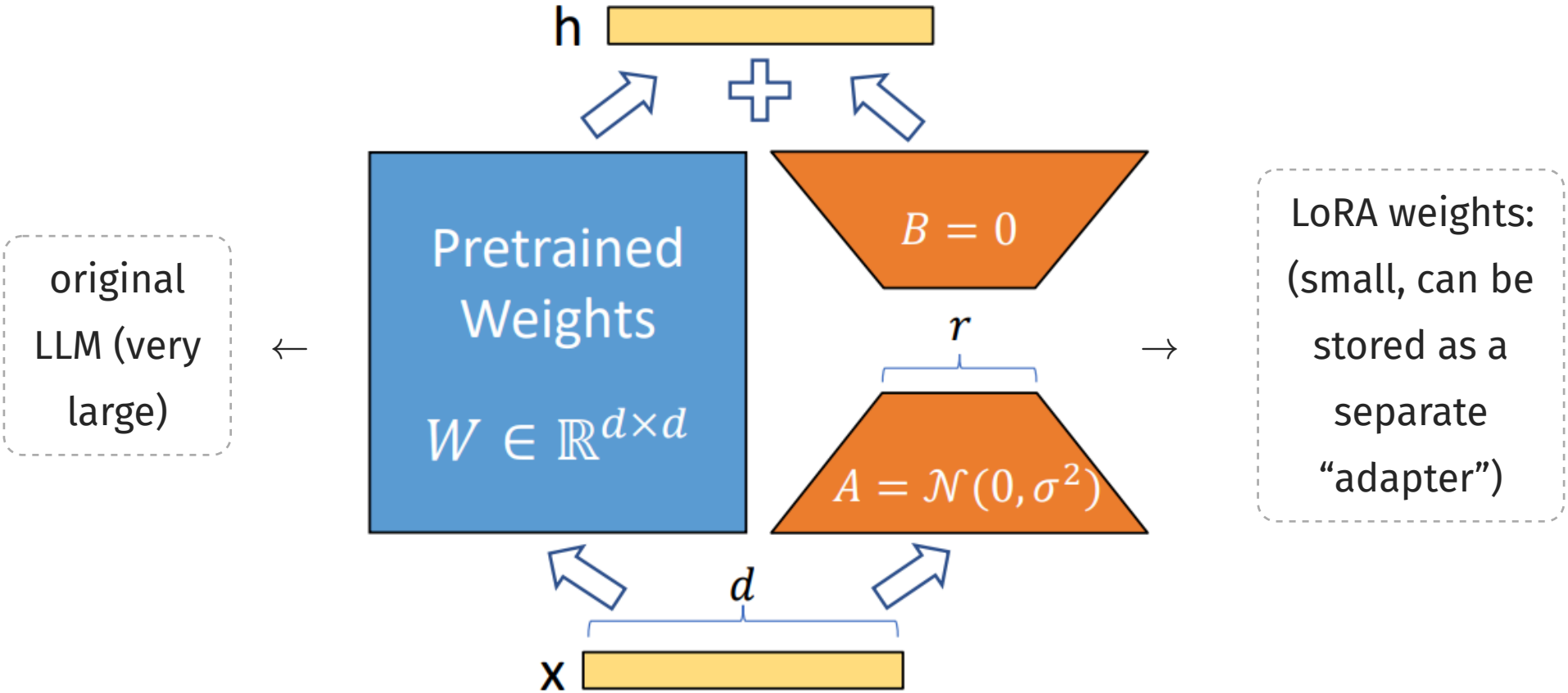
Key ideas of LoRA

- Instead of updating model weights W directly, we keep the ΔW (the "diff") in a **separate matrix**.
- We ensure that the ΔW matrix has a **low rank** → it can be decomposed into two smaller matrices.

$$W' = W + \Delta W = W + BA$$

where $W \in \mathbb{R}^{d \times k}$, $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

Low-rank adaptation (LoRA)



Low-rank adaptation (LoRA)

source: Hu et al. (2021)

- The original model weights W are **frozen** (no gradient updates), only the small matrices A and B are trained.
- For inference, LoRA weights can be **merged** into the base model:

$$W' = W + BA$$

  → no additional latency at inference time.
- The LoRA weights can be stored as a separate **adapter** (typically a few MB).

Info

For a rank $r = 8$ and $d = 4096$: LoRA adds only $2 \times 4096 \times 8 = 65,536$ parameters per layer instead of $4096^2 \approx 16.7M$.

QLoRA: finetuning quantized models

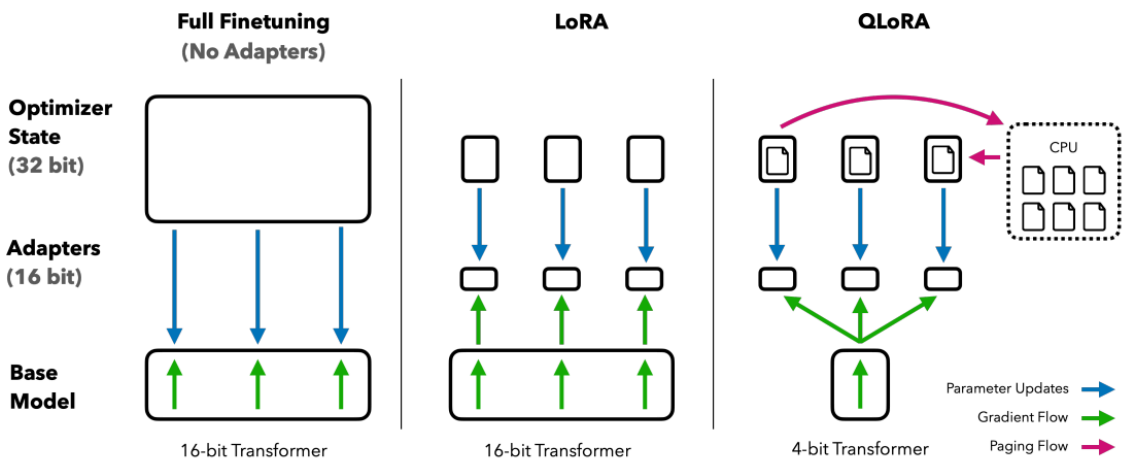
source: Dettmers et al. (2023)

Idea

Can we combine **quantization** and **LoRA** for even higher efficiency?

QLoRA (Dettmers et al., 2023):

- Careful parameter quantization + optimizing CPU-GPU memory transfers.
- Enables finetuning a 65B model on a single 48 GB GPU.



LoRA and QLoRA in practice

All things being equal: **full finetuning > LoRA > QLoRA** in terms of model performance.

→ The goal is to make the most out of the memory you have.

Method	Bits	Memory (7B model)
Full finetuning	32	≈ 120 GB
Full finetuning	16	≈ 60 GB
LoRA	16	≈ 16 GB
QLoRA	8	≈ 10 GB
QLoRA	4	≈ 6 GB

source: LLaMA-Factory

Frameworks: LLaMA-Factory, HuggingFace PEFT, Unsloth.

Other PEFT methods

LoRA is not the only parameter-efficient finetuning method:

- **Prefix tuning** (Li & Liang, 2021): prepend learnable "virtual tokens" to the input.
- **Prompt tuning** (Lester et al., 2021): similar to prefix tuning, but only for the input layer.
- **Adapters** (Houlsby et al., 2019): insert small bottleneck layers into each Transformer block.
- **DoRA** (Liu et al., 2024): weight-decomposed LoRA – decomposes weight updates into magnitude and direction.

In practice

LoRA and QLoRA remain the **most popular** PEFT methods by far, thanks to their simplicity and strong performance.

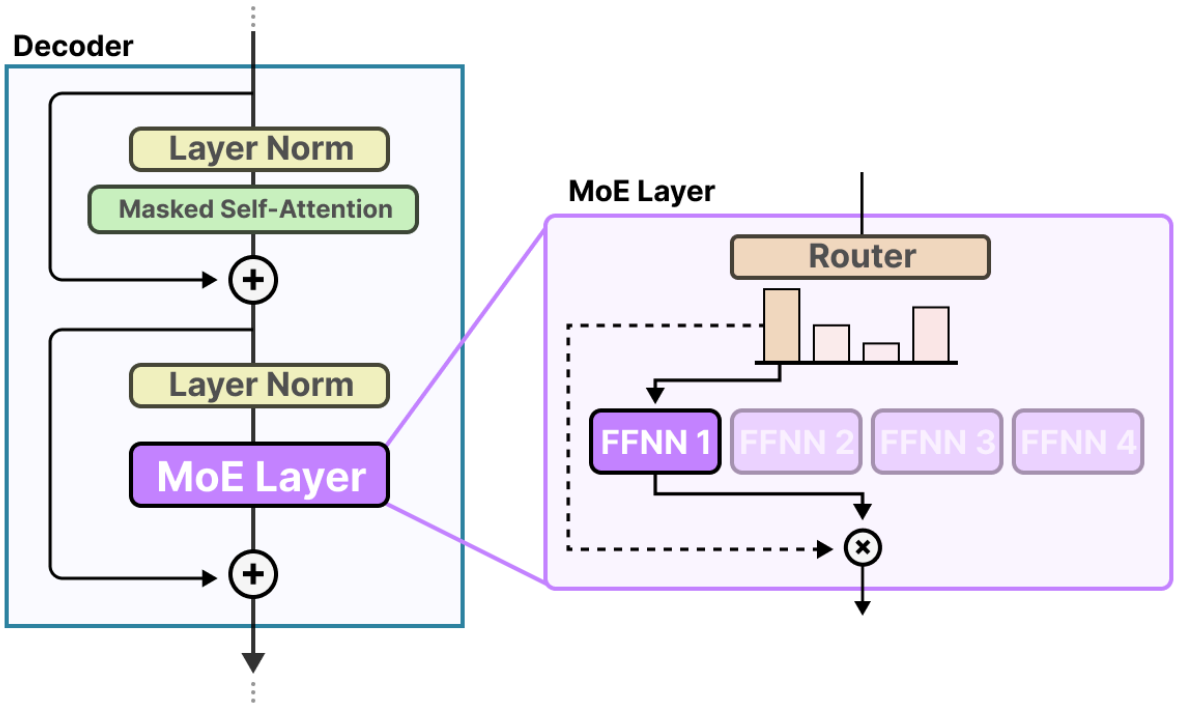
Mixture of experts

Mixture of experts (MoE)

source: Grootendorst (2024)

Idea

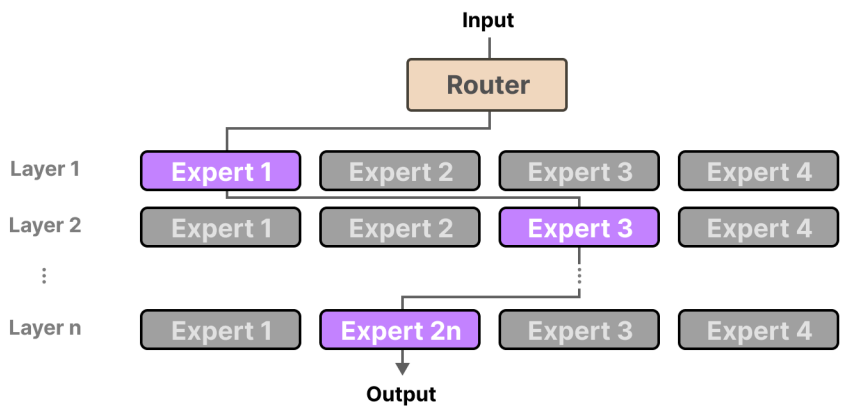
Replace the single feed-forward network (MLP) in each Transformer layer with **multiple experts**. A **router** selects which experts to use for each token.


Decoder
Layer Norm
Masked Self-Attention
Layer Norm
MoE Layer
MoE Layer
Router
FFNN 1
FFNN 2
FFNN 3
FFNN 4

MoE routing

source: Fedus et al. (2024)

We can choose a **different expert at each layer** and a **different set of experts for each token**:



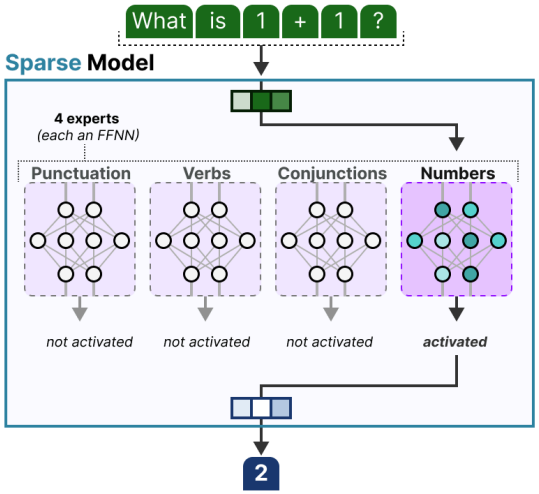

```
class MoeLayer(nn.Module):
    def __init__(self, experts: List[nn.Module],
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args
```

Why MoE works

source: Grootendorst (2024)

Why is this a good idea?

- Individual experts can **specialize** to solving certain kinds of problems.
- **Faster training** for the same number of total parameters (we only backpropagate through selected experts).
- **Faster inference** (although we still need to load the full model into memory).



Example: Mixtral 8×7B → 8 expert MLPs, approximately equivalent to a **47B dense model**

(not 56B, since attention layers and embeddings are shared).

The router

source: Grootendorst (2024)

The **router** (or "gating network") is a small learned network that decides which experts to activate:

$$G(x) = \mathrm{softmax}\big(\mathrm{TopK}\big(x \cdot W_g\big)\big)$$

- W_g is a learnable weight matrix.
- **TopK**: only the top K experts (typically $K = 1$ or $K = 2$) are activated.
- The router outputs **weights** for combining expert outputs.

⚠️ **Load balancing**

If the router always picks the same experts, some experts are underutilized. MoE models use **auxiliary losses** to encourage balanced expert utilization.

MoE models in practice

MoE has become a very popular architecture for recent LLMs:

Model	Total params	Active params	Experts
Mixtral 8×7B	46.7B	12.9B	8 (top-2)
Mixtral 8×22B	176B	39B	8 (top-2)
DeepSeek-V3	671B	37B	256 (top-8)
Qwen3-235B	235B	22B	128 (top-8)
GPT-4 (rumored)	$\approx$ 1.8T	$\approx$ 220B	16 (top-2)

Key insight

MoE models can be much larger in **total** parameters while keeping the **active** parameter count (and thus inference cost) comparable to smaller dense models.

MoE: tradeoffs

Advantages

- Faster training and inference for a given quality level.
- Experts can specialize.
- Scales well with more experts.

Disadvantages

- **Memory**: the full model must be loaded (all experts), even though only a few are active.
- **Load balancing**: uneven expert utilization wastes capacity.
- **Communication overhead**: in distributed settings, tokens need to be routed across devices.

Question

MoE models are often paired with **quantization** to address the memory issue.

Why does this make sense?

Efficient attention

Linear attention

source: Schoelkopf (2024)

Standard attention has $O(N^2)$ complexity due to the softmax over all pairs of queries and keys:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(QK^T\right)V$$

Very expensive similarity computation

$$\text{Softmax}\left(QK^T\right)V = \frac{\exp(QK^T)}{\sum_{i=1}^{L} \exp(QK_i^T)} V$$

Standard attention formula

Question

Can we find a cheaper way to compute attention?

Linear attention: rearranging the computation

source: Schoelkopf (2024)

We can replace softmax with a cheaper kernel function φ:

$$\mathrm{Attention}(Q, K, V) = \varphi(Q)(\varphi(K)^\top V)$$

This allows us to **rearrange** matrix multiplications:

Standard: $QK^\top \in \mathbb{R}^{N \times N}$

$\rightarrow O(N^2 \cdot D)$

Linear: $K^\top V \in \mathbb{R}^{D \times D}$

$\rightarrow O(N \cdot D^2)$

The resulting operations are $O(N \cdot D^2)$, which is **linear in sequence length**.

Linear attention: does it work?

source: Schoelkopf (2024)

In theory, linear attention is very attractive (linear vs. quadratic).

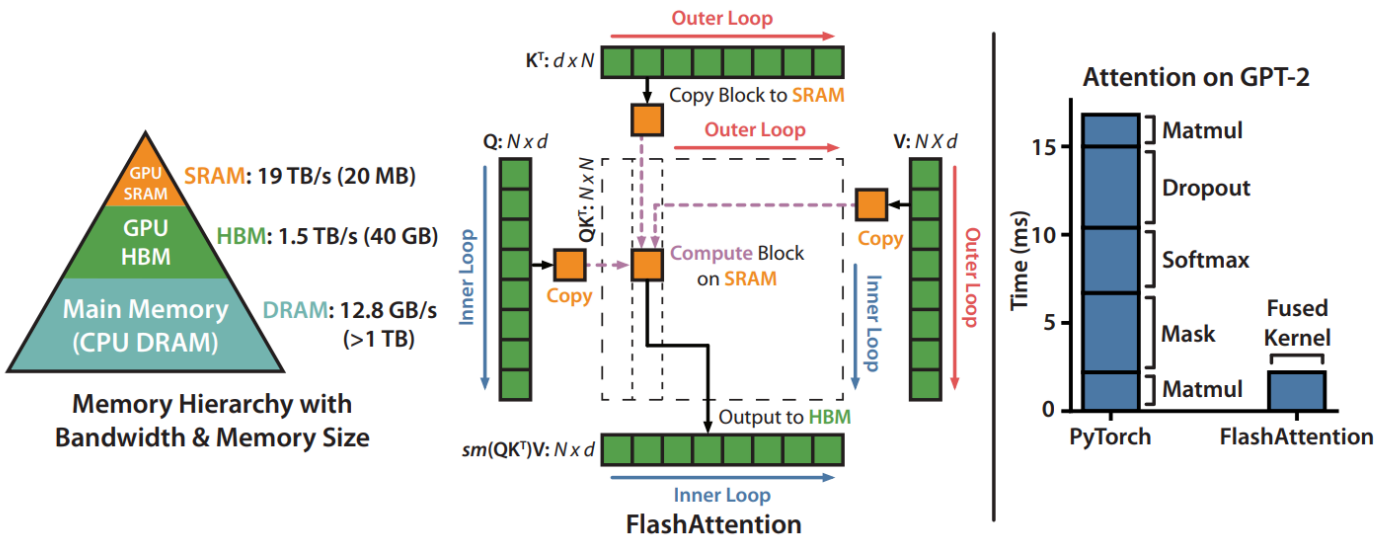
In practice, performance **degrades** compared to full softmax attention.

Recent models like **Mamba** (Gu & Dao, 2023) and **RWKV** use variants of linear attention with additional tricks to close the gap.

FlashAttention

source: Dao et al. (2022)

Approximations like linear attention can degrade performance. Meanwhile, **hardware optimizations** of the full attention mechanism can go a long way.



FlashAttention: how it works

source: Dao et al. (2022)

FlashAttention re-arranges the attention computation to better use the **GPU memory hierarchy**:

- Uses **tiling** to break the attention computation into small blocks that fit in fast GPU SRAM.
- Avoids materializing the full $N \times N$ attention matrix in slow GPU HBM.

Properties:

- **Fast**: 2–3× faster than standard attention.
- **Memory-efficient**: linear in sequence length.
- **Exact**: uses no approximations (unlike linear attention).

- Now **commonly implemented** in all major LLM frameworks (PyTorch, HuggingFace, vLLM, ...).

Summary

Summary

Quantization

- Reduce precision (float16 → int8 → int4).
- Huge memory savings with small accuracy loss.
- GPTQ, GGUF for practical deployment.

Distillation

- Train a small student from a large teacher.

LoRA / QLoRA

- Freeze base model, train low-rank adapters.
- Dramatic reduction in finetuning memory.
- Can be combined with quantization (QLoRA).

- Soft labels transfer more knowledge.
- White-box vs. black-box approaches.

Mixture of experts

- Multiple expert MLPs, only a few active.
- More capacity without proportional compute.
- Used in Mixtral, DeepSeek, Qwen3.

Next lecture

Questions?